Chapter 1
GNU/Linux Operating System

Abstract In this introductory part we describe the basic GNU/Linux system usage and the various tools and utilities which are used with it. In particular, the use of the GNU/Linux command-line, Bash shell scripting, and external programs such as `find`, `tar` etc. are explained. Secure communication tools OpenSSH and VNC are also described. Since open-source software is written using computer programming languages we describe the commonly used open-source programming languages, including C/C++, FORTRAN, Ada, Java, Python, Tcl/Tk, Perl, Common Lisp, Scheme, Erlang, Smalltalk, Ruby, Scala, X10, and Lua in this chapter. Example for each of these languages and their salient points are described.

Contents

1.1	**Basic GNU/Linux Usage**	**5**
1.2	**Bash shell**	**11**
1.3	**External commands and programs in GNU/Linux**	**12**
1.4	**Next steps**	**15**
1.5	**OpenSSH: OpenBSD Secure Shell**	**18**
1.6	**Programming Languages**	**19**
1.7	**Miscellaneous Topics**	**34**
1.8	**Conclusion**	**34**

We focus on the fundamentals of any Open Source system for Scientists and Engineers. Since this book focuses on the usage of computer tools, the Operating System (OS) which forms the primary interaction layer with the user, is particularly important. We have written the book primarily for a POSIX[1] compliant UNIX like system; but many of the applications described in this book have been ported to other OS as well.

In Section 1.1 we discuss the GNU/Linux operating system and its relevance to open source tools for scientists and engineers.

Besides the OS, the compiler used for generating executable binaries from the source code also has a fundamental impact on the quality, performance and portability of the application. Especially for Open Source tools, where availability of the

[1] Portable Operating System Interface for UNIX, IEEE 1003.

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_1, © Springer Science+Business Media, LLC 2011

source code is a given, knowing the intricacies of the compiler can be very beneficial. In Section 3.1 we talk about GCC (GNU Compiler Collection) compilers.

GNU/Linux (basically all UNIX derived OS) share a number of common features and facilities. This include:

1. Files: the OS has several types of files, such as regular files, directories, devices, symbolic links, named pipes, and sockets. Interaction with kernel, and devices takes place using special device files. Similarly, a *program* is a binary file residing on a file-system file which is *loaded* into memory, combined with other shared libraries, to form an executable object which the kernel then schedules as a process,
2. Multi-user: the OS has user id (`uid`), group id (`gid`), and process id (`pid`). These are used for *accounting*, where the OS keeps tracks of system usage based on user accounts, and group quotas,
3. Resource permissions: GNU/Linux is one of the most secure operating systems (when configured correctly), and modern technologies such as SE Linux have improved the authentication and security even more. Every resource has permissions set on its use, be it files, directories, and executables. An user can only perform the action with the resource if the user id has sufficient privilege level. This prevents misuse of the system, and guards against malicious attempts to gain access,
4. Multi-tasking: since its conception GNU/Linux was a multi-tasking OS, but multi-core computers have only recently become commonplace (so much so, that now in 2010 it is difficult to mind a single core CPU). By design GNU/Linux has support for multiple processes, and multi-threading. The scheduler in the Linux kernel has been enhanced to improve the response time on regular desktop workloads, while the multi-CPU SMP kernel is already used in many supercomputers with large number of CPUs. In addition to supporting multiple CPUs, the Linux kernel also has excellent support for inter-process communication using shared memory, named pipes (FIFOs), and sockets. All POSIX features for IPC are supported, and Linux also supports extensions which are not in POSIX,
5. System calls: being POSIX compliant, Linux kernel supports all the POSIX system calls. A system call is a facility or functionality provided by the kernel to the application program through a well defined interface. The use of system calls provides a layer of abstraction which enables a well written program to be ported from one POSIX OS to another with minimal change. System calls in GNU/Linux are provided with a C language API, and are similar in form to the function APIs provided by the C library. For example a program may register a cleanup function to be called when the application program exits. An example is shown in Listing 1.1.

```
/** \file example_atexit.c
    \author Sandeep Koranne (C) 2010
    \description Example of using the atexit function
*/
#include <stdio.h>            /* program IO */
#include <unistd.h>           /* POSIX */
#include <stdlib.h>           /* atexit */
```

```
static void CleanupAfterwards(void) {
  fprintf( stdout, "%s: ", __FUNCTION__ );
  fprintf( stdout, "Deleting application files...\n");
}

int main( int argc, char* argv[] ) {
  long max_atexit = sysconf( _SC_ATEXIT_MAX );
  int rc;
  if( max_atexit == 0 ) perror(" atexit..\n" );
  rc = atexit( CleanupAfterwards );
  if( rc != 0 ) perror( "atexit failed..\n");
  return (0);
}
```

Listing 1.1 Using the `atexit` function

`atexit` may even be registered for dynamic libraries, to be called when they are unloaded,

6. Sysconf functionality: the system configuration of the currently running system can be introspected using the `sysconf` functionality. Portable programs, or application programs that attempt to retune their runtime parameters based on system configuration can inspect the state of the system. Some parameters, such as *page size* can have a large impact on the performance of the program. The most common system configuration arguments are given in Table 1.1.

Table 1.1 POSIX.1 variables for `sysconf`

Variable	Description
`ARG_MAX`	maximum length of arguments to `exec` functions,
`CHILD_MAX`	maximum number of simultaneous process per user id,
`HOST_NAME_MAX`	maximum length of a hostname,
`LOGIN_NAME_MAX`	maximum length of login name,
`_SC_CLK_TCK`	number of clock ticks per second,
`OPEN_MAX`	maximum number of open files per process,
`PAGESIZE`	size of page in bytes,
`_SC_PHYS_PAGES`	number of pages of physical memory,
`_SC_AVPHYS_PAGES`	number of pages of memory currently available,
`_SC_NPROCESSORS_CONF`	number of processors configured,
`_SC_NPROCESSORS_ONLN`	number of processors online.

1.1 Basic GNU/Linux Usage

Linux (Figure 1.1 shows the official Linux mascot) is the most popular open source operating system. Technically, Linux refers to the UNIX-like kernel written by Linus Torvalds, but most users don't interact with the computer's kernel, instead they rely on a host of other software which runs on top of the kernel. It is for this rea-



Fig. 1.1 Tux, the Linux mascot

son that the operating system described in this book is referred to as GNU/Linux. It includes the kernel, the C runtime library, GNU utilities and tools, shells, and graphical windowing systems.

Today GNU/Linux runs on mainframes, personal computers, and even embedded systems such as cellphones and gaming consoles. GNU/Linux is also a favorite development platform for open source systems relevant to scientists and engineers, and is the epitome of the open source movement. This book was designed and authored on a number of GNU/Linux systems, using open source tools.

The advantages of GNU/Linux can be summarized as follows:

1. Linux has a stable and efficient kernel which is optimized (and undergoes constant improvement),
2. Complete source code of the kernel and system utilities is available,
3. Its POSIX compliant and thus easy for application developers to port,
4. Since it has the UNIX philosophy, it is resilient against malicious software (to a large extent), has good security model, is multiuser, and separates the graphical windowing system from the underlying OS,
5. GNU/GCC and other high quality development tools are readily available,
6. More and more high-performance supercomputers are running GNU/Linux, and thus systems such as MPI, OpenMP, cluster computing are first available on GNU/Linux.

GNU/Linux is often available as a *distribution* which is a packaged, tested and complete operating system. The distribution can be commercial (open source does not imply free-of-cost), or it can be free (for example Fedora or Ubuntu). But all GNU/Linux systems are fundamentally the same, the differences mostly are in the application packaging, updates, and themes. Some distributions tend to focus on vertical segments such as education, electronics. There is also a distribution dedicated for scientists and engineers, but many of the software tools are available on all distributions, or can be compiled on the available distribution. The source code of the kernel can be found in `/usr/src/kernels` if the kernel development package was installed.

1.1.1 System Calls

An useful tool to find out which system calls are being issued by a program is `strace`, which intercepts the system calls made by its argument program and prints them either to standard error or to a file (with the -o option). Running `strace uptime` gives the following data on a Linux 2.6.31 system:

```
execve("/usr/bin/uptime", ["uptime"], [/* 86 vars */]) = 0
brk(0)                                  = 0x816c000
mmap2(NULL, 4096, PROT_READ|PROT_WRITE
...
access("/etc/ld.so.preload", R_OK)
mprotect(0xaa7000, 8192, PROT_READ)     = 0
mprotect(0x934000, 4096, PROT_READ)     = 0
munmap(0xb787d000, 133998)              = 0
uname({sys="Linux", node="celex", ...}) = 0
open("/proc/stat", O_RDONLY|O_CLOEXEC)  = 3
read(3, "cpu  230207 235 59156 1265206 32"..., 8192) = 2856
close(3)                                = 0
time(NULL)                              = 1277426073
brk(0)                                  = 0x816c000
brk(0x818d000)                          = 0x818d000
open("/etc/localtime", O_RDONLY)        = 3
...
open("/proc/uptime", O_RDONLY)          = 3
lseek(3, 0, SEEK_SET)                   = 0
read(3, "119176.07 12652.06\n", 2047)   = 19
..
open("/proc/loadavg", O_RDONLY)         = 4
lseek(4, 0, SEEK_SET)                   = 0
read(4, "0.77 1.00 1.17 2/301 5624\n", 2047) = 26
write(1, " 17:34:33 up 1 day,  9:06,  4 us"..., 69) = 69
exit_group(0)                           = ?
```

Even without consulting the code for the `uptime` utility we can garner that it produces its output by reading various system files in `/proc`, namely, `/proc/stat` and `/proc/loadavg`.

1.1.1.1 Usage Limits

The functions `getrlimit` and `setrlimit` are used to get and set resource limits respectively. These functions prevent a single rogue process on the machine from consuming all of the available resources (such as CPU time, memory, files, etc). Using sensible limits on the resource in your programs is advised. Consider the small program in the listing below:

```
////////////////////////////////
// limitexample.cpp
// (C) Sandeep Koranne, 2010
// Example showing how to put limits on resources
////////////////////////////////
```

```
#include <iostream>
#include <sys/resource.h>
#include <stdio.h>
void AllocateMemory() {
  while( true ) {
    int *n = new int[1024];
    if( !n ) return;
  }
}

int main( ) {
  struct rlimit mem;
  mem.rlim_cur = mem.rlim_max = 1024000;
  int rc = setrlimit( RLIMIT_AS, &mem );
  if( rc ) perror("setrlimit:");
  AllocateMemory();
  return( 0 );
}
```

Listing 1.2 Example of using resource limits

By using the `setrlimit` function we cap the maximum memory usage by this program to 10240 bytes. Without this usage restriction, the program would go into swap, and the whole system becomes unresponsive, until the operating system kills the offending program, which could be a long time as the virtual memory system is implemented using disk (which is many orders slower than physical RAM). Thus, specifying reasonable limits, even then they are computed at runtime (based on input file size), makes for a robust application software. An application program can inspect the current *environment* of the user using the `getenv` function. This provides a simple way of allowing the user to customize parameters to the program. An example is shown in Listing 1.3.

```
/** \file example_getenv.c
    \author Sandeep Koranne, (C) 2010
    \description Example of using getenv
*/
#include <stdio.h>            /* for program IO */
#include <stdlib.h>           /* for getenv */
#include <string.h>           /* for strcmp */

int main( int argc, char *argv[] ) {
  enum { NO_ALGO=0, LINEAR, BINARY };
  int which_algo = NO_ALGO;
  const char* use_algo = getenv( "USE_ALGORITHM" );
  if( use_algo == NULL ) which_algo = NO_ALGO;
  else if( !strcmp( use_algo, "LINEAR" ) )
    which_algo = LINEAR;
  else if( !strcmp( use_algo, "BINARY" ) )
    which_algo = BINARY;
  else {
    fprintf(stderr, "Unknown algorithm : %s\n", use_algo );
    which_algo = NO_ALGO;
  }
  printf("Using algorithm %d\n", which_algo );
  return (0);
}
```

Listing 1.3 Example of `getenv` function

Running this program gives us:

```
./example_getenv
```

```
Using algorithm 0
$ export USE_ALGORITHM="BINARY"
$ ./example_getenv
Using algorithm 2
$ export USE_ALGORITHM="RANDOM"
$ ./example_getenv
Unknown algorithm : RANDOM
Using algorithm 0
```

1.1.1.2 Signals

Signals in GNU/Linux (and UNIX) are event notifications. The nomenclature has roots in hardware implementations of control, and even today, actions such as key-press interrupt of Ctrl-C sends a signal to the application program. The computation itself may raise a signal from the CPU such a arithmetic divide-by-zero. Signals can be responding to an external condition, or based on timer intervals (alarm signal). The key concept to be noted for signals is that they are asynchronous, and a program using signals should be cognizant that a signal can arrive at any time, and during any computation. A common use of signals is to intercept the SIGINT signal (which is the result of Ctrl-C) to perform cleanup actions. We should note that `atexit` does not cleanup when the program terminates due to signal. An example of intercepting Ctrl-C (or the equivalent SIGINT key on the OS) is shown in Listing 1.4.

```
/** \file example_signal.c
    \author Sandeep Koranne, (C) 2010
    \description example of using SIGNALS in GNU/Linux
*/
#include <stdio.h>            /* program IO */
#include <signal.h>           /* for sigaction */
#include <stdlib.h>           /* for exit */
#include <unistd.h>           /* for _exit */
#include <string.h>           /* for memset */

static void SecondChance( int signal_number ) {
  int yes_or_no;
  fprintf( stderr, "Are you sure ?\n" );
  fscanf( stdin, "%d", &yes_or_no );
  if( yes_or_no ) _exit( 0 );
}

int main( int argc, char *argv [] ) {
  int i;
  struct sigaction action;
  memset( &action, 0, sizeof( sigaction ) );
  action.sa_handler = SecondChance;
  if( sigaction( SIGINT, &action, NULL ) ) {
    perror( "sigaction failed....\n" );
    exit( 1 );
  }
  while( 1 ) ;
  return 0;
}
```

Listing 1.4 Example of signal handling in GNU/Linux

For an user a GNU/Linux is like any other multiuser UNIX system with login shells, hierarchical file systems, text configuration files, and UNIX development tools. The shell (which is the program responsible for interaction with the user) supports job control, scripting, input redirection and other facilities. In the next section we have discussed briefly the Bash shell.

1.1.2 GNU/Linux introspection

GNU/Linux has a number of system commands which can be used to gather information about the system. These commands can be useful when discussing or reporting problems about particular software installations, or runtime problems. We discuss some of these commands below:

- uname: with the -a option prints detailed information about the system, as:

  ```
  Linux celex 2.6.31 Wed Dec 9 11:14:59 EST 2009 i686 GNU/Linux
  ```

  with the -m option prints shorter machine identification which can be used in compiling machine specific code,
- /proc/cpuinfo: is a /proc file which contains detailed information about the number and type of CPU(s) in the machine. The instruction set of the CPU can be checked to see if it has SIMD or advanced instructions,

  ```
  processor : 0
  vendor_id : GenuineIntel
  cpu family : 6
  model : 13
  model name : Intel(R) Pentium(R) M processor 1.60GHz
  stepping : 6
  cpu MHz : 600.000
  cache size : 2048 KB
  ...
  ```

- /proc/meminfo: similarly contains information about installed memory, also see the `free` command.

1.1.3 GNU coreutils

For any GNU/Linux system there is a set of command known as *coreutils*, these can be thought of as fundamental user commands which perform common and useful actions. We list some of these below:

- man: format and display the manual pages. This command can be used to find more details about the commands listed below. In fact, `man man` gives you information about `man` itself,

- Commands to list output of files: includes `cat`, `od`, and `nl`,
- Formatting: `fmt`, `fold`,
- Output parts of files: `head`,`tail`, and `split`,
- Summarizing files: `wc`, `sum`, `cksum`, `md5sum`, and `sha1sum`,
- Sorting: `sort` and `uniq`,
- Operating on fields within a line: `cut`, `paste`, and `join`,
- Operating on characters: `tr`, and `expand`,
- Directory listing: `ls`, and `dir`,
- Basic operations: `cp`, `mv`, `rm`, `shred`, `dd`, and `install`,
- Changing file attributes: `chown`, `chmod`, `chgrp`, and `touch`,
- Disk usage: `df`, `du`, `stat`, `truncate`, and `sync`,
- Printing text: `echo`, `printf`, and `yes`,
- Conditions: `false`, `true`, `test`, and `expr`,
- Redirection: `tee`,
- File name manipulation: `basename`, `dirname`, and `pathchk`,
- Working context: `pwd`, `tty`, and `printenv`,
- User information: `id`, `logname`, `whoami`, `groups`, `users`, and `who`,
- System context: `date`, `arch`, `uname`, `hostname`, `hostid`, and `uptime`,
- Process control: `kill` and `sleep`.

1.2 Bash shell

The Bash shell is an UNIX shell for the GNU system and is often the default shell on GNU/Linux. Bash is actually an acronym for *Bourne-again shell*, which is a pun on the *Bourne* shell. Bash is POSIX compliant, but also has a number of extensions. We list some of the important features of Bash below:

- Brace expansion: running:

```
[skoranne@celex ~]$ echo a,{b,c,d},e
a,b,e a,c,e a,d,e
```

- Keyboard shortcuts: supports Emacs style keyboard shortcuts,
- Startup scripts: loads and executes `/etc/profile` for login, `/.bashrc` for interactive shells,
- Redirection: supports input and output redirection.

1.2.1 Bash shell scripting

The Bash shell supports scripting and programming. This feature is very useful in automating repetitive tasks and system administration. Consider a directory with JPG files, and that you want to add a prefix to all the filenames, such as “Printed”.

If there are hundreds of files doing such a task by hand is tedious, and more importantly, un-necessary on a system like GNU/Linux running Bash.

```
$for i in $(ls *.jpg);do
  mv $i "Printed_$i"
done
```

Consider a real life example:

```
for i in $(ls *.png);do
  echo "Converting image file $i"
  convert $i -colorspace Gray $(basename $i .png)_bw.png
done
```

This simple program will do the required action. Using the `basename` command, we can separate a file name from its designated extension (.png) in this case. The external command `convert` is then called with the two file names, the input and the output (which has the suffix 'bw'). The conversion is a grayscale conversion. In addition to the above mentioned `for-do-done` loop, there are also `if-then-else` statements.

A more detailed example of using Bash scripting to perform an experiment is shown in Listing 1.5.

```
#!/bin/bash
# Sandeep Koranne, (C) 2010
#
# Shell script to sweep configuration parameters for OpenMP
# we run the program multiple times with different options
# and varying the THREAD count

# Scalability plot for program on 2-CPU
CMD=./collatz_omp
for num_threads in $(seq 1 2); do
export OMP_NUM_THREADS=$num_threads
NUM_TH=$(echo $OMP_NUM_THREADS)
for algo in $(seq 0 5);do
echo "Running algorithm $algo"
datFileName="$algo"_"$NUM_TH".dat
echo "Data goes in $datFileName"
for i in $(seq 19 21);do
echo "Running N = " $i
/usr/bin/time -f "%e %U" -o $datFileName -a $CMD $i $algo
done
done
done
```

Listing 1.5 Example of using Bash scripting

1.3 External commands and programs in GNU/Linux

In addition to the GNU coreutils commands, there are a number of external commands which are readily available on GNU/Linux and perform many useful tasks.

The regular expression feature is common to many of these commands, and it is instructive to list the regular expression syntax.

1.3.1 GNU regular expression syntax

A regular expression is built up from blocks that match single characters, or patterns. Single characters are patterns which match themselves, any meta-character can be matched by preceding it with a backslash. The '.' character (period) matches any single character. Characters listed inside *brackets* form a list, from which any single character can be matched. For example:

```
[school]
[^home]
```

the first pattern list matches any character in 's', 'c', 'h', 'o', and 'l'; the second list has a `'^'` symbol as the first character, implying that characters *not* in the list can be matched.

Character ranges can be specified by a hyphen. A + after the list bracket implies one or more repetition of that class can be matched, while implies a zero or more match. For example:

```
[0-9]+ matches any non-empty number
[a-z]* matches any word, even empty
[0-9]{9} matches 9 numbers
```

Now we discuss some useful commands below:

1. `wget`: utility to download files from the Internet in a non-interactive manner. It supports HTTP, HTTPS as well as FTP. `wget` can work in the background, as well as resume downloading partially downloaded files. It can also be used for recursive downloads of sites. Another related program is `curl`, which is used to transfer a single URL (see an example of using `curl` in Section 20.8),
2. `find`: GNU find searches the directory tree routed at the given path, example:

   ```
   find /home/skoranne -name '*.cpp'
   find /home/skoranne -name '*.tar' | xargs ls -l
   ```

   `find` can search for files based on their type, date of modification, etc. Actions can also be specified for files which match the pattern,
3. `grep`: program to find regular expression based pattern in files, or standard input. Some of the important options for `grep` are:

   - `-v`: negate the matching,
   - `-m`: specify number of matches per file,
   - `-A`: print number of lines after context,
   - `-B`: print number of lines before context,
   - `-w`: match complete words only,

- `-i`: case independent matching,
- `-n`: print line number with match,
- `-H`: print file name for each match.

4. `sed`: Sed is a stream editor. A stream editor is used to perform basic text transformations on an input stream, for example:

```
cat A.txt | sed -e 's/alpha/beta/g' > B.txt
```

5. `awk`: GNU awk is an implementation of the *awk* language. The most common use of `awk` is to parse an input file (line by line) into fields. Consider the following example:

```
$cat A.txt
ABC  WV   65
DEF  SJ   87
GHI  MN   18
$cat A.txt | awk '{print $1}'
ABC
DEF
GHI
$cat A.txt | awk '{ printf "City " $2  " # " $3 "\n"}'
City WV # 65
City SJ # 87
City MN # 18
```

   There are a number of other features in `awk`, and a complete list of commandline options and features can be found by running `info awk`,
6. `rsync`: remote file copying and synchronization tool. This tool can be used to keep two computer's contents in sync, as is the case when mirroring a website (for example), or transfering documents from one workstation to another,
7. `ssh`: is an OpenSSH client and remote login program. SSH has rapidly become the remote login program of choice since it is much more secure and flexible than `telnet`, and `rlogin`. It provides secure and encrypted communication capability between two untrusted hosts over an insecure network. It can also be used to *tunnel* other protocols such as VPN, X11 and VNC (see Section 1.7.1). `ssh` can be configured to use *authentication keys* in lieu of passwords. A related program is *secure copy*, `scp`. A more detailed description of OpenSSH is given in Section 1.5,
8. `nc`: `nc` (short form of netcat) is a TCP/UDP analysis tool. It can open TCP/UDP connections, send and listen on ports and deal with IPv4 as well as IPv6,
9. `screen`: GNU screen is a full-screen window manager that multiplexes a physical terminal between several processes. When `screen` is first called it creates a single window with a shell. To switch between terminals a key combination can be used, such as `Ctrl-a n`. A new window can be created using "C-a c". Using `screen -x` a remote second user can attach to a non-detached screen session (which is a way to share a session),
10. `xargs`: the `xargs` programs reads input separated by blanks, and executes the specified command for each input token. `xargs` is used in combination

with `find` or other file name listers to perform system administration and disk cleanup tasks. For example, to check the time stamp for all files which match the regular expression `m[a-e]k` we can use the following *oneliner*[2]

```
$ ls | grep -i m[a-e]k | xargs ls -l
  2915 2010-06-27 01:26 make.aux
 19103 2010-07-31 22:54 make.tex
 19099 2010-07-26 00:27 make.tex~
```

In the above example we chained the output of `ls` to `grep` where we used regular expression matching. As we know, `grep` will output only those patterns which match the expression, and these are passed to `xargs`; the command to be executed by `xargs` is `ls -l`, which prints the required time stamp information. `xargs` can also take the input from a file (in addition to the standard input),

11. `tar`: GNU `tar` (acronym for Tape Archive) is used to collect directories and files into a single file for archival on tape, or backup. It supports a number of command-line options, the most common ones are:

```
-c, --create               create a new archive
-r, --append               append files to the end of an archive
-t, --list                 list the contents of an archive
-x, --extract, --get       extract files from an archive
-p, --preserve-permissions, --same-permissions
-f, --file=ARCHIVE         use archive file or device ARCHIVE
-C, --directory=DIR        change to directory DIR
-v, --verbose              verbosely list files processed
```

Thus a new archive to contain all files in directory XYZ can be done as `tar cvf xyz.tar XYZ/`. Moreover, the archive can be compressed using the following compression schemes. The compression schemes included in GNU `tar` are:

a. gzip: using the `z` option,
b. bzip2: using the `j` option,
c. lzma: using the `J` option,

12. `diff`: the `diff` command can be used to find differences between files. Especially for source code, `diff` is used for version control management and change control.

1.4 Next steps

As familiarity with GNU/Linux grows, the user can write useful shell scripts, or *one liners*, to perform administration work. The software systems presented in the remainder of this book assumes that the user is able to operate the operating system to perform the input, output, and file system tasks. Before closing this chapter let me

[2] A oneliner in UNIX terminology is a combination of UNIX utilities and shell constructs, which is created to solve a particular problem, and is often comprised of only one line.

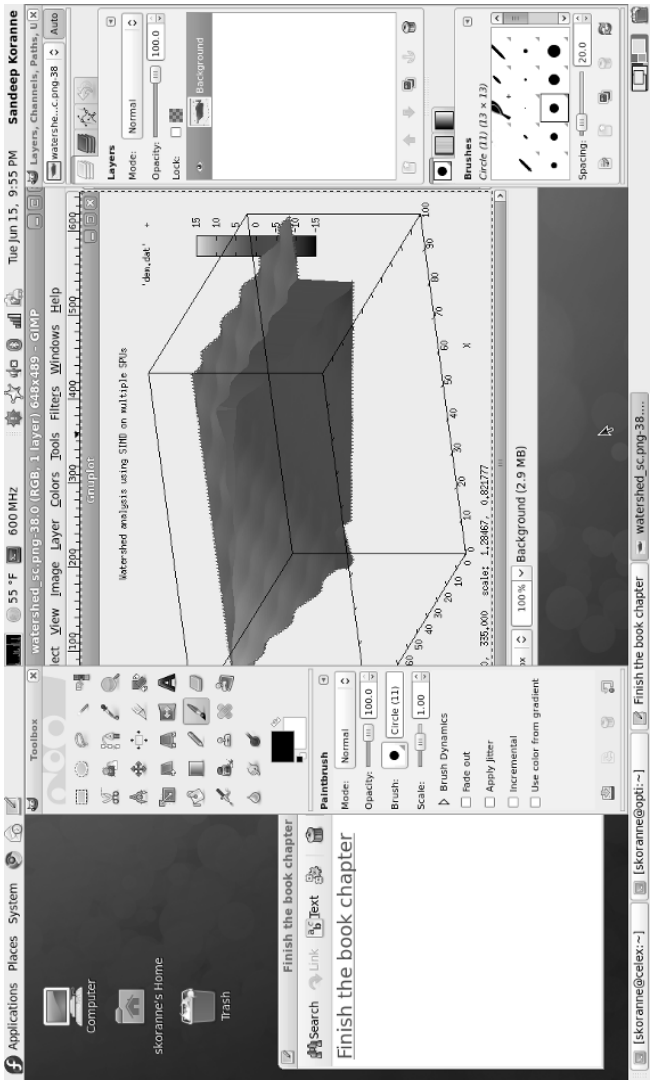
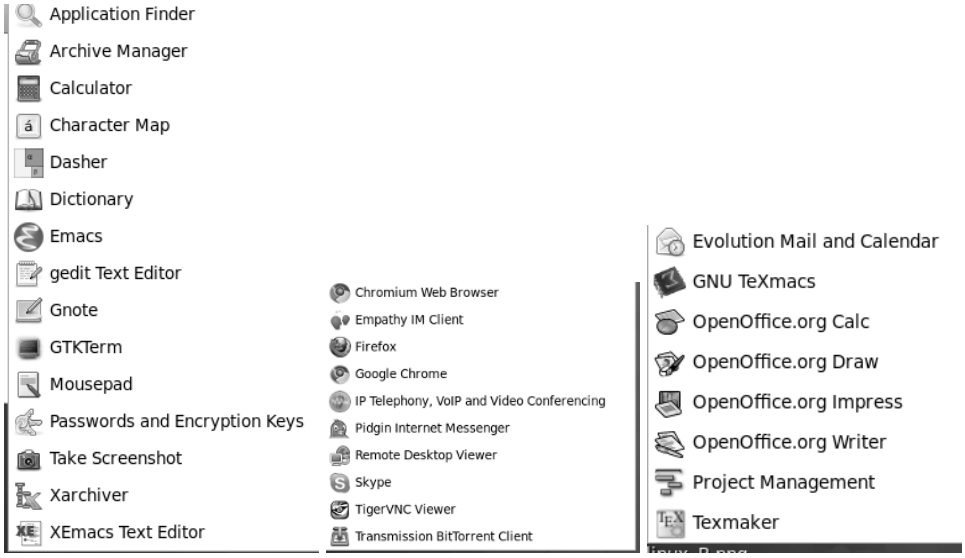


Fig. 1.2 GNU/Linux desktop showing GNU Image Processing Tool (GIMP)

present the graphical user interface of GNU/Linux as this may be the more common interaction paradigm for most users today.

Consider the GNU/Linux screenshot of a computer running Fedora 12 as shown in Figure 1.2. The graphical system is GNOME, and the system menus have a number of applications for tasks such as system administration, office applications, Internet and networking, and software development. These are shown in Figure 1.3. I am not listing programs such as Firefox, Thunderbird since these are in popular use.



(a) Applications, Internet and Office



(b) Software development and System

Fig. 1.3 GNU/Linux with GNOME Menus

1.5 OpenSSH: OpenBSD Secure Shell

OpenSSH (OpenBSD Secure Shell) is an open source implementation of the SSH connectivity tools. As compared to Telnet, Rlogin, and FTP (which transmit their password and all user data in clear text on the network), SSH encrypts all traffic. This eliminates the risk of snooping and hijacking, as well as other attacks. OpenSSH also provides tunneling capabilities over which other protocols such as X11, and VNC can be used with the same security advantages.

OpenSSH is comprised of a number of tools:

1. sshd: the SSH services daemon (can be compared to 'telnetd' daemon),
2. ssh: rlogin and telnet like login client,
3. ssh-keygen: key generator tool,
4. ssh-config: daemon configuration file,
5. scp: secure copy program (like rcp).

1.5.0.1 The `ssh` client

The `ssh` client is the primary login and remote execution client for SSH. The usual manner of using `ssh` is:

```
$ssh [options] remote-server
```

When logging into a remote system, `ssh` supports a number of command line arguments; the most common and useful ones are listed below:

- `-C`: requests compression of data using zlib compression,
- `-c`: selects cipher for encryption, supported ciphers include *blowfish*, des,
- `-F` :specify alternative configuration file,
- `-f`: request ssh to go background just before command execution,
- `-i`: specify identity file,
- `-L`: specifies port forwarding,
- `-l`: specify remote login name,
- `-o`: specify option,
- `-p`: specify port on remote server,
- `-v`: verbose mode (useful for debugging a connection problem),
- `-X`: enables X11 forwarding,
- `-Y`: enables trusted X11 forwarding.

`ssh` exits with the exit status of the remote command or 255 if an error occurred.

Prior to a successful connection, `sshd` verifies that the client machine is a trusted party (by checking the name of the host in `/etc/hosts.equiv`), and that the user names are same on both machines. Moreover the server must be able to verify the client's host key. The file `/.ssh/authorized_keys` on the remote server lists the public keys that are permitted to login. The `ssh` program on the client informs the server of the key pair it is going to be using for the session, and then the `sshd` server checks whether the corresponding public key is authorized.

The user can create a key-pair by running `ssh-keygen`. If using OpenSSH protocol 2 with DSA the generated keys are stored in the file `/.ssh/id_dsa`, when using OpenSSH protocol 2 with RSA the generated keys are stored in the file `/.ssh/id_rsa`. The corresponding public keys (which must be transmitted to the server and placed in `/.ssh/authorized_keys`) are generated in `/.ssh/id_dsa.pub` (for DSA) and `/.ssh/id_rsa.pub` (for RSA). Once the public keys are placed on the server, the `ssh` client (and any program which uses `ssh`) can login to the remote server without giving the password. Although, if the public keys do not exist, then `ssh` falls back to password based login.

The `ssh` client also maintains a database of identifications of all hosts it has connected to; if the host key changes for some reason (or more commonly if the host IP address changes), `ssh` warns about this change.

External tools such as CVS (see Section 3.2.2) and Subversion SVN (see Section 3.2.3) also use SSH to perform remote version control through remote repositories.

1.6 Programming Languages

Since this book is about open-source software, and all software is written in a computer programming language, it behooves us to describe some of the languages and their implementations which are available as open-source systems themselves. In this section we discuss some of the popular programming languages available on GNU/Linux system. Particular programming syntax for each language is not presented, however the particular strength of the language and its application domain is mentioned in case the reader wants to investigate the use of a particular language for a specific task.

1.6.1 C and C++

C and C++ are used as system programming, as well as application programming languages. Many of the libraries presented in the sequel of this book (see Chapter 4) are written in C/C++. C is especially popular for system programming, writing system kernels and tools. The function calling conventions of C language are sufficiently common so as to become the de-facto standard for new languages. Consequently, a number of existing libraries in C can be easily used from other language using *foreign function interface*.

A number of compilers for C and C++ are available. These include the GNU GCC Compiler suite. GCC is discussed in depth in Section 3.1. LLVM (low level virtual machine) (discussed in Chapter 13, section 13.6) also implements a C and C++ compiler `clang`. Open64 group also has an open-source compiler for C and C++. In recent years, GCC has made promising strides in optimization, better in-

struction generation on modern processors, automatic parallelization, compiler optimizations and error checking, threading support, and support for OpenMP. The upcoming C++ standard is also supported by GCC's C++ compiler and standard library.

1.6.2 GNU FORTRAN

On GNU systems, there exist a number of FORTRAN compilers and source translators, including `g77`, `f2c` and GNU FORTRAN (part of GCC). GNU FORTRAN has support for FORTRAN 95 and uses the same back end as GCC C/C++ compilers thus benefiting from portability and integration with libraries written in C and C++. GNU FORTRAN can be run using the `gfortran` command and accepts many of the same general command line arguments as GCC (see Section 3.1 for a detailed list of GCC's options). GNU FORTRAN accepts the OpenMP directives as specified in OpenMP version 3.0. See Section 12.2 for a detailed explanation of OpenMP support and programming. An example of a short FORTRAN program to print the number of OpenMP threads is shown in Listing 1.6.

```
! \file fomp.f
! \author Sandeep Koranne, (C) 2010
! \description Example of using OpenMP in GNU Fortran
      PROGRAM FOMP
      IMPLICIT NONE
      INTEGER OMP_GET_MAX_THREADS
      INTEGER OMP_GET_THREAD_NUM
      write( 6, "(a,i3)") " MAX THREADS : ", OMP_GET_MAX_THREADS()
      write( 6, "(a,i3)") " Thread num  : ", OMP_GET_THREAD_NUM()
      END PROGRAM
```

Listing 1.6 GNU FORTRAN example with OpenMP

We can compile this program as:

```
$gfortran fomp.f -o fomp -lgomp
$./fomp
 MAX THREADS :  16
 Thread num  :   0
```

1.6.3 Ada

The GNU GCC compiler collection also has a compiler for the ADA programming language. Consider the simple hello, world style program in Ada.

```
-- this is hello world in Ada
with Text_IO; use Text_IO;
procedure HelloWorld is
begin
```

```
   Put_Line( "Hello from Ada!" );
end HelloWorld;
```

We create a file `helloworld.adb` with these content, then we can compile this Ada program using the GNU gnat system.

```
$gnatmake helloworld.adb
gcc -c helloworld.adb
gnatbind -x helloworld.ali
gnatlink helloworld.ali
$./helloworld
Hello from Ada!
```

Later (in Section 15.6) when we discuss the VHDL hardware description language, its similarities to ADA will be apparent.

1.6.4 Java

Although many Java software development kits are available on GNU/Linux, we discuss the GNU GCC compiler suite's `gcj` Java compiler. GNU `gcj` is an *ahead-of-time* compiler for the Java language. Using the `-C` option `gcj` can generate the Java bytecode, otherwise it will generate native instructions (for the architecture GCC was compiled for). Consider the small Java program in Listing 1.7.

```
// \file hw.java
// \author Sandeep Koranne, (C) 2010
// \description Hello world in Java compiled with GNU gcj

class hw {
    public static void main( String args[] ) {
  System.out.println(" Hello, World!\n" );
    }
}
```

Listing 1.7 Compiling Java program with GNU `gcj`

The Java language has restrictions on the names of classes present in files, for example the file 'hw.java' has `class hw` which has the `main` function. Compiling this file using the GNU `gcj` compiler we get:

```
$gcj --main=hw hw.java
$ldd a.out
linux-gate.so.1 =>  (0x00f80000)
libgcc_s.so.1 => /lib/libgcc_s.so.1 (0x00603000)
libgcj.so.10 => /usr/lib/libgcj.so.10 (0x0381f000)
```

Compiling the file using the `--main` command line argument specifies the class which contains the `main` function. This creates an executable file which is linked with the `libgcj` runtime library. We can also compile the file to Java bytecode using the `-C` argument as:

```
$gcj -C hw.java
$ls -l hw.class
                    413 2010-07-16 20:09 hw.class
$java hw
Hello, World!
```

The bytecode is executed through the Java Virtual Machine.

1.6.5 Python

The Python programming language implemented on most GNU/Linux is the CPython reference implementation. The version used for many of the examples in this book is Python version 2.6.2. Many of software libraries discussed in this book have been integrated with Python. And even complete tools such as SConstruct (see Section 3.4.2) have been constructed around Python. Due to its interactive nature Python has also been used to develop glue software integrating diverse tools into a complete package. In fact the SAGE mathematical software system has been written in Python (see Subsection 17.14), and is developed as a collection of several independent math software tool, all connected using Python.

Although Python is a general purpose programming language, some features of Python are especially suited for high-performance scientific computing and solving engineering problems.

1.6.5.1 NumPY

NumPy is a Python package developed for scientific computing with Python. It contains:

1. N-dimensional array: *n*-dimensional array object,
2. sophisticated broadcasting functions
3. tools for integrating C/C++ and FORTRAN code
4. linear algebra, Fourier transform: integration with external library codes for linear algebra, FFT and random number generation.

As an example of NumPy consider:

```
Python 2.6.2 (r262:71600, Aug 21 2009, 12:22:21)
[GCC 4.4.1 20090818 (Red Hat 4.4.1-6)] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> from numpy import *
>>> a = array( [ 1,2,3,4])
>>> a
array([1, 2, 3, 4])
>>> b = array( [5,6,7,8] )
>>> a+2*b
array([11, 14, 17, 20])
```

In the above listing, the use of NumPy arrays as well as functions operating on array elements can be observed.

NumPy is used effectively with SciPy (a package for scientific computing with Python). These packages can be imported in Python as:

```
>>> import numpy as np
>>> import scipy as sp
>>> import matplotlib as mpl
>>> import matplotlib.pyplot as plt
```

The `matplotlib` is a general purpose plotting library for Python. SciPy contains many mathematical functions including optimization, linear interpolation, special functions and more. Consider an example shown below:

```
import numpy as np
from scipy import interpolate
import matplotlib.pyplot as plt
x = np.arange(0,20)
y = np.exp(-x/3.0)
f = interpolate.interpolate1d(x,y)
xnew = np.arange(0,20,0.1)
plt.plot(x,y,'o',xnew,f(xnew),'-')
plt.title(``Example of interpolation'')
plt.show()
```

The resulting plot is shown in Figure 1.4.

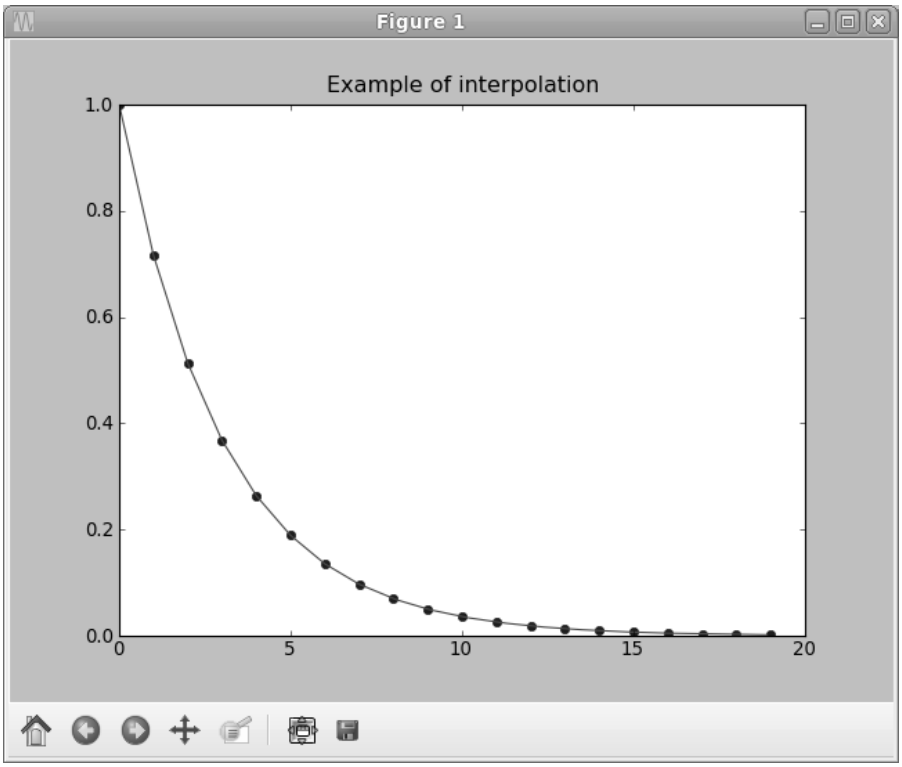


Fig. 1.4 Example of matplotlib in Python

Matplotlib has many other features which are shown in Section 19.1.5.5, where we discuss building graphical user interfaces (GUI) with Python and Qt.

1.6.5.2 PyCUDA

Python has well designed facilities for multi-processing and multi-threading. An example of calling external processes through Python is shown in Section 19.1.5.5. In this section we describe a more recent development in the parallel programming arena, GPGPU (general computing on graphics processing units). A leading software library for GPGPU computing is nVidia's CUDA software. PyCUDA is a Python module which makes the integration with CUDA simple. Consider the following example:

```
import numpy as np
import pycuda.driver as cuda
cuda.init()
assert cuda.Device.count() >= 1
dev = cuda.Device( 0 )
contxt = dev.make_context()
func = mod.get_function(``myf'')
a_on_cpu = np.arange(0,10)
a_on_gpu = cuda.mem_alloc( a_on_cpu.nbytes )
cuda.memcpy_htod( a_on_gpu, a_on_cpu )
func( a_on_gpu, block=(1,1,1) )
cuda_memcpy_dtoh( a_on_cpu, a_on_gpu )
```

1.6.5.3 PuLP

SciPy has a number of general-purpose optimization routines including: (i) Nelder-Mead Simplex algorithm, (ii) Quasi-Newton method, and (iii) Minimize sum-of-squares. But for general linear problems (LP) formulated using variables and constraints, or combinatoric problems modeled as integer linear programs (ILP) the Python LP modeler (PuLP) can be used. Once the model has been defined in PuLP a number of linear solvers can be called to solve the problem. See Subsection 16.10 for more details on PuLP, however a simple example is presented below:

```
x = LpVariable(``x'',0,10)
y = LpVariable(``y'',5,20)
problem = LpProblem(``example'', LpMinimize )
prob += x + 3*y <= 20
prob += 2*x+y
status = prob.solve( GLPK( msg = 0 ) )
LpStatus[status]
```

The above listing creates a LP problem with two variables x and y which has one constraint, $x+3y \leq 20$, while the objective function is $2*x+y$ and the goal is to *minimize* the LP. The solver chosen in GLPK (see Section 16.9 for more details on GNU GLPK and other linear solvers).

1.6.6 Tcl/Tk

Tcl (Tool control language) and Tk GUI Kit are interactive programming languages which were designed for embedding into applications. Tcl, however, is a general purpose programming language which is used for rapid prototyping and has been used to develop complete applications (see TkCVS in Section 3.2.5). Tk is the GUI toolkit which is used to design and deploy GUIs written in Tcl and other languages (such as Python using the Tkinter library). Tcl is designed to be extensible by C, C++ and other Tcl modules. All expressions in Tcl are written as *commands* which are variadic and written in prefix notation. Example:

```
% set sum [expr 1+2]
3
% puts "Hello"
Hello
pack [button .b -text "Hello"]
```

Tk provides a number of *widgets*, including: (i) buttons, (ii) message-box, (iii) toolbars, and (iv) drawing canvas. Tcl/Tk are commonly used in providing scripting capabilities in engineering applications; however, Python is also being used in more modern applications for the same purpose.

1.6.7 Perl

Perl is a general purpose, high-level computer programming language. It was originally designed for efficient and easy report generation and processing. Its design was heavily influenced by C, shell scripting, AWK, and `sed`. Perl's handling of regular expressions and capabilities to process large amounts of textual data, combined with its rapid use in initial CGI scripting for the Internet caused its exponential rise. Perl is often used to automate system administration tasks which are too complex for shell scripting, while not compute intensive enough to warrant a full C/C++ application. Moreover, Perl has integrated support for hash-tables, regular-expressions, string support, and file-handling on most operating systems. With Perl version 5.0, the language was optimized with a new interpreter, lexical variables, and modules. Extension modules to Perl (often written in Perl, but possibly in C) are available for many tasks such as XML processing. Modules are available on CPAN (Comprehensive Perl Archive Network). It is easy to recognize Perl code since it uses symbols to distinguish major data types. Fundamental data types included in Perl are: (i) scalars ($), (ii) array (), and (iii) associative array (%). Other data-types are: (iv) globs, (v) file-handles, and (vi) sub-routines. Implementation of Perl on GNU/Linux are version 5.10.0. A short example of Perl code using an associative array is shown below:

```
#!/usr/bin/perl -w
my %days = ("mon" => 1, "tue" => 2, "wed" => 3 );
```

```
foreach my $day (keys %days) {
    print "$day has $days{$day}\n";
}
```

1.6.8 Common Lisp

Common Lisp is general purpose programming language which is best associated with the development of large and complex applications dealing with symbolic logic and artificial intelligence. Symbolic math software (such as Maxima, see Section 17.1) are usually written in Common Lisp. Lisp is an acronym of List Processing, although the Common Lisp language has first class support for many data-structures including arrays, hash-tables, strings, and streams. Common Lisp was one of the first standardized languages to have a complete object-oriented (OO) methodology as part of the language. The Common Lisp Object System (CLOS) is an advanced OO framework which supports inheritance and message dispatch. Lisp (like Python) has an interpreter (called the REPL (read-eval print loop)), but Common Lisp implementations have sophisticated compilers which often produce object code rivaling C and FORTRAN in their number crunching speeds. Lisp also introduced the concept of *garbage collected* storage, and dynamic type. Open-source implementations of Common Lisp include GNU Common Lisp (clisp), CMU Lisp, and SBCL. Consider a Common Lisp function as shown in Listing 1.8.

```
(defun compute-function ( a b )
  (if (= 0 (logxor a b)) -1 1))
(defun ck (A n k)
  (let ((ans 0)(temp_j 0)
        (n_temp_k 0)(ub (- n k )))
    (dotimes (j ub)
      (setf temp_j (aref A j))
      (setf n_temp_k (logxor (aref A (+ j k)) 1))
      (incf ans (compute-function temp_j n_temp_k)))
    ans))
(defun cs (A n)
  (let ((sum 0))
  (dotimes (i (- n 1))
    (let ((val (ck A n (1+ i))))
      (incf sum (* val val))
      (format t "~%c_~D = ~D" (1+ i) val)))
  sum))
(cs #(1 0  1 0 1) 5)
(quit)
(defun merit-factor (A)
  (let ((n (length A)))
    (* 1.0 (/ (* n n) (* 2 (cs A n))))))
(defun log2 (n)
  (dotimes (j 64)
    (when (> (ash 1 j) n) (return-from log2 j))))
(defun eb (n pos)
  (let ((SHIFT (1- (log2 n))))
    (ash (logand
    n (ash 1 (- SHIFT pos)))
   (- pos SHIFT))))
```

Listing 1.8 Common Lisp functions

1.6.9 Scheme

A closely related language to Common Lisp is the Scheme language which shares many of the functional programming roots with Lisp. GNU's Scheme implementation is called Guile as it is the official scripting language for application extension of the GNU project. The idea behind using interactive scripting languages for application extension comes from separating the compute intensive portion of the application and writing it in a language such as C or C++, while the bulk of the application can be written (and customized, even by the end-user) using a language such as Scheme. Towards this end, GNU Guile has a simple integration method where Guile can be linked with an application and used as the scripting interface. The wrapper generator SWIG can be used to automate this process; more details on using SWIG for wrapper interface generation are given in Section 9.7. Python, Tcl and Lua are also commonly used for application scripting interface. An example of Scheme code is:

```
(define sum (lambda (a b ) (+ a b)))
guile> (sum 1 2)
3
```

Scheme, although superficially similar to Lisp, had (prior to Common Lisp, atleast) fundamental differences surrounding the rules for variable scoping (lexical scoping for Lisp was first introduced in Scheme), namespaces for functions and implementation of iterative loops. Even today, Scheme is generally regarded as an elegant and more research oriented language than Common Lisp, which arguably has many more features.

1.6.10 Erlang

On GNU/Linux, the Erlang implementation can be run using the `erl` command-line which starts the Erlang shell:

```
Erlang R13B04 (erts-5.7.5) [source] [rq:1]
                           [async-threads:0]
                           [kernel-poll:false]

Eshell V5.7.5  (abort with ^G)
1> % this is a comment
1> 3+3
1> .
```

```
6
```

The advantages of using Erlang are manifest when programming on multi-processor machines, or when writing applications for fault-tolerance, as Erlang has *single assignment variables*.

```
X = 1234.
1234
3> X = 5678.
** exception error: no match of right hand side value 5678
```

In the above example the variable *X* was assigned the value 1234; then later when we try to assign another value 5678 to *X*, Erlang raises an exception. Since a variable can be set only once in Erlang, the debugging of programs as well as analyzing the program for execution on a parallel machine is simplified (as Erlang has no mutable state in variables, it does not need shared memory locking).

Erlang has enumerated types called *atoms* defined by the use of lowercase variable names, and a fixed number of items can be collected into a single entity using *tuples*. However, unlike C language `struct`, tuples in Erlang are anonymous, so it is common to use an atom as the first element in a tuple to denote its intended use in the program:

```
Point = {point, {x,10}, {y,20}}.
{point,{x,10},{y,20}}
8> Point.
{point,{x,10},{y,20}}
```

Erlang uses *pattern matching* operator to satisfy the equality operator:

```
{point,CX, CY} = Point.
{point,{x,10},{y,20}}
13> CX.
{x,10}
14> CY.
{y,20}
{another, DX, DY } = Point.
** exception error: no match of right hand
   side value {point,{x,10},{y,20}}
```

By checking the pattern `{point,X,Y}` against the `Point` tuple, we assign the variable *CX*,*CY* to the corresponding elements in the tuple, as the first element in both LHS and RHS is `point`. In the second example, when we try to match a pattern with `another` as the first element, we get an exception.

Erlang has *lists* which are very similar to Common Lisp lists:

```
AL = [1,2,3,4,5].
[1,2,3,4,5]
17> [CAR_AL| CDR_AL ] = AL.
[1,2,3,4,5]
18> CAR_AL.
```

```
1
19> CDR_AL.
[2,3,4,5]
```

The `[X|Y]` notation denotes the *head* and *tail* of the list respectively, and like all Erlang pattern matchings, can be used to extract the head and tail of the list respectively.

Functions can be defined in Erlang as shown below. The listing is placed in a file called 'number_theory.erl':

```
%% \file number_theory.erl
%% \author Sandeep Koranne, (C) 2010
%% \description Examples in Erlang, for number theory

-module( number_theory ).
-export( [calc_collatz/1] ).
-export( [fibonacci/1] ).

%% Actual functions
calc_collatz( {collatz,1 } ) -> 1; % simple case
calc_collatz( {collatz,X } ) when ( X rem 2 ) == 0
      -> calc_collatz( {collatz,X div 2} );
calc_collatz({collatz,X}) -> calc_collatz({collatz,3*X+1}).

fibonacci(0) -> 0;
fibonacci(1) -> 1;
fibonacci(X) -> fibonacci( X-1 ) + fibonacci( X-2 ).
```

The *module* declares the `number_theory` module. It *exports* the two functions `calc_collatz` and `fibonacci`, both accepting a single input (the *arity* of the function is part of the function declaration in Erlang). We can bring this module into a running Erlang shell by:

```
c(number_theory).
{ok,number_theory}
>number_theory:fibonacci(6).
8
```

Similar to Perl, Erlang also has a Comprehensive Erlang Archive Network (CEAN) which attempts to gather the major applications for Erlang in one common place. Erlang is commonly used in designing fault-tolerant communication software. An example of fault-tolerant software is CouchDB (see Section 20.8 for information).

1.6.11 Smalltalk

Smalltalk-80 is a general purpose programming language which introduced many concepts in programming language design such as *object oriented design*, *messages*,

and the *model-view-controller* method of GUI design. Smalltalk-80 is also noted for its use of a workspace environment for interacting with the user, as compared to the syntactical text in, object code out view of programming languages. Smalltalk-80 is implemented on GNU/Linux system using the Squeak software system. A picture of an actual Smalltalk-80 session in GNU/Linux is shown in Figure 1.5.

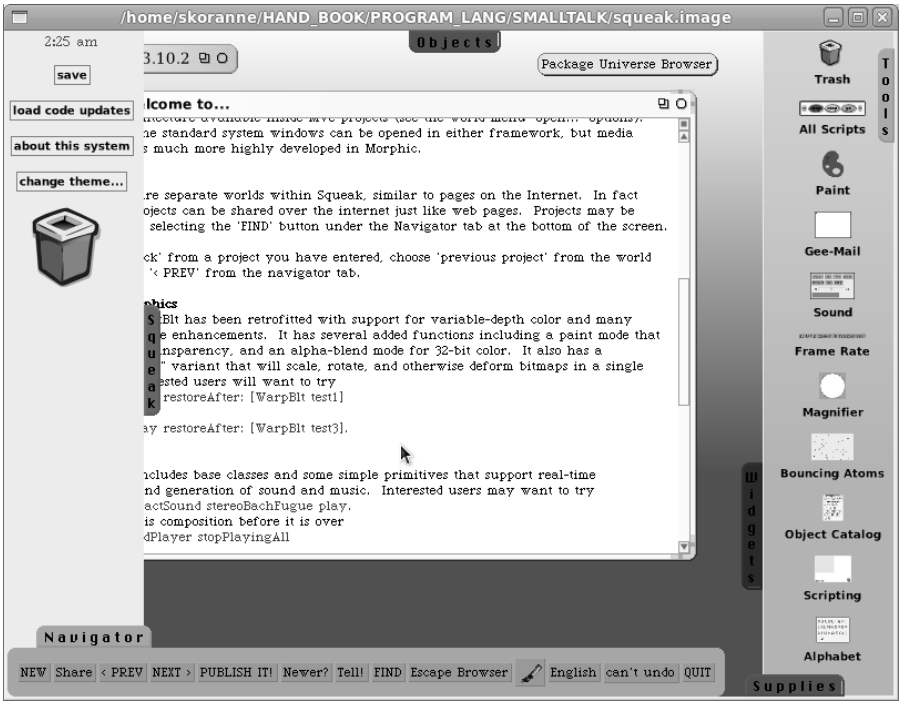


Fig. 1.5 Smalltalk-80 programming workspace implemented in Squeak on GNU/Linux

1.6.12 Scala

The design of the Scala programming language started as a research project to develop better language support for component software, where components can be libraries, modules, classes, frameworks, processes, or even web services. As such, Scala is by design an amalgamation of object-oriented and functional language design, with the over riding goal of *scalability*, that it should be possible to describe large as well as small systems using the same concepts. Scala has been implemented on the Java Virtual Machine on GNU/Linux, and Scala programs resemble Java programs in many ways. However, Scala introduces the concept of singleton class objects using the `Object` keyword. Scala also does not distinguish between statements and expressions, and every function returns a value.

Scala allows arguments to be passed by name, which can be used to implement *short-circuiting* as below. Consider the following user-defined class in Scala:

```
abstract class Boolean {
 def && (x: => Boolean): Boolean;
 def || (x: => Boolean): Boolean;
}
```

The use of `=>` in passing the formal argument allows the parameter to be passed without evaluation into the function.

Functions in Scala are values (they can be passed as first-class objects) as well as Objects. The `map` method applies a given function to every object in a sequence. Scala's library defines many types of sequences including: (i) arrays, (ii) lists, (iii) streams, and (iv) iterators. Like Python, Scala too has *list comprehensions* which can succinctly represent code which operates on elements of a sequence:

```
def sqrts(xs: List[double]): List[double] =
  for( val x <- xs; 0 <= x ) yield Math.sqrt(x)
```

Here, the constraint $0 \leq x$ acts as a filter to remove negative values from the *xs* list of doubles. The values which pass through this filter are used as a generator and passed to the variable *x* which is then *yielded* to form the return sequence (which is also of type List[double]).

Scala has been used to create interactive and high traffic sustaining websites which require scalability to 100 million or more transactions per hour. In addition to the facilities described in this short section, Scala has other features such as pattern recognition, compositions and XML processing which are also very powerful.

1.6.13 Google's GO Programming Language

Go is an expressive, concurrent, garbage-collected systems programming language developed by Google. Consider the following canonical example:

```
// this is a simple Go Program
package main
func main() {
  print("Hello, World! from Go\n")
}
```

We can compile this program using the `8g` Go compiler on an i386 machine running GNU/Linux as:

```
$ echo $GOOS
linux
$ echo $GOARCH
386
$ ../bin/8g -h
gc: usage: 8g [flags] file.go...
```

```
$ ../bin/8g -V
8g version 5917
$ ../bin/8g helloworld.go
# this produces the file helloworld.8 file
# which can be linked using the 8l linker
$ ../bin/8l -h
usage: 8l [-options] [-E entry] [-H head] [-L dir]
          [-T text] [-R rnd] [-r path] [-o out] main.8
$ ../bin/8l helloworld.8
# which produces the file called 8.out
$ ./8.out
Hello, World! from Go
```

The Go programming language is already being used for system programming tasks.

1.6.14 X10 Language

As parallel programming on a large scale (clusters of millions of computers) becomes a reality, the existing programming paradigms of shared-memory, or message-passing have shown the limits of their scalability. Towards this end, a new language, the X10 Programming Language, has been designed with the concept of partitioned global address space (PGAS). X10 is a modern object-oriented programming language in the PGAS family with the goal of enabling scalable and high-performance programming for next-generation computer systems.

In addition to supporting PGAS semantics, X10 introduces asynchronous actions and distinguishes between the two views of synchronous execution (defined as executing within a *place*) and asynchronous execution. Each *place* runs lightweight *activities* which are synchronous and atomic in the memory space of the place they execute on. Furthermore, data is classified as immutable (in which case no consistency management is necessary, and it can be freely copied) or data consistency management may require one or more *clocks* to order the execution. The array datatype supports parallel collective operation on data.

X10 is object oriented, and like C++ programmers write X10 code by defining containers for data and behavior called *interfaces*, e.g.:

```
interface Diameter {
   def compute_diameter() : Double;
}
class SimpleGraph implements Diameter {
   var x : Double = 0;
   public def isConnected() { }
   public def compute_diameter() { }
}
struct DirectedGraph implements Diameter {
   public def compute_diameter() {}
}
```

In the above listing, both `SimpleGraph` as well as `DirectedGraph` implement the `Diameter` interface, in that they both implement the `compute_diameter` method which returns a `Double`. When the programmer states that `SimpleGraph` implements `Diameter`, it must implement all the method that the interface demands.

In X10 there is a distinction between **`class`** and **`struct`**, **`struct`** are headerless values and cannot have mutable fields, like the **`class`** `SimpleGraph` has field *x*. The advantage of **`struct`** is the methods can be inlined; moreover, **`struct`** are immutable. Although, X10 has no primitive class, the X10 standard library defines classes for: (i) Boolean, (ii) Byte, (iii) Short, (iv) Integer, (v) Long, (vi) Float, (vii) Double, (viii) Complex, and (ix) String. X10 is also a functional language, with functions being first-class objects (functions can be stored and passed to other functions as values). The GNU/Linux implementation of X10 (which is still considered a research language) is available and implements version 2.0 of the X10 language specification.

1.6.15 Lua

Lua is a general purpose programming language designed to be used as a scripting language for application extension. It is a lightweight language, and is used in extending game development engines and providing interactive control to users of the application. In many respects, Lua is similar to Scheme (see Section 1.6.9) as both languages are used for application extension and scripting. Lua is a dynamically typed language and supports a small number of fundamental data-types, including: (i) Boolean, (ii) numbers, and (iii) strings. The native data structure in Lua is the *table* from which other data-structures such as list, set, arrays, and records can be devised. The Lua table is basically an associative array. Like Scheme, Lua also has support for *first class functions*, garbage collection, closures, proper tail recursion, and co-routines. The use of Lua tables is essential to programming in Lua, consider the following program fragment:

```
person={name="Jack", age=15,
        marks={physics=80, math=75, english=70}}
print(person)
table: 0x84bc578
print(person.marks.physics)
80
```

Tables in Lua are always passed by reference, so if a table is assigned and modified, both tables are modified. Tables can be used to implement record structures as shown in the above example.

Lua is used by video game developers as a scripting language due to its small embedded footprint, fast execution speed, and relatively small learning curve. As an application extension language Lua competes with Scheme and Python.

1.7 Miscellaneous Topics

In addition to the programming languages discussed above, software implementation can be done using database processing languages such as SQL (structured query language) (see Chapter 20 for a discussion on open-source database systems).

1.7.1 VNC : Virtual Network Computing

In Section 1.5 we saw the use of OpenSSH for tunneling X11 connections securely over SSH protocol. VNC provides additional facility for remote X11 connections. It allows for tunneling using SSH through a trusted intermediary node (often a designated VPN gateway), allows for sharing of the display (allowing collaborative work), and persistent display workspace which can be connected from any machine.

VNC comprises of two tools, the VNC server `vncserver` and the VNC client viewer `vncviewer`. The server has to be launched on the remote machine and is setup to ask for a connection password. The display size, and color depth can be specified on the `vncserver` command-line. The display then starts off with a display number 1 larger than the maximum X11 display. To connect to a VNC server we can use the `vncviewer` application. The viewer application also has a number of command-line options such as: (i) full-screen mode, (ii) JPEG quality level, (iii) gateway tunnel server, (iv) shared display, (v) viewonly, and (vi) zlib compression level. Another remote desktop sharing application on GNU/Linux is `rdesktop` (remote desktop).

1.8 Conclusion

In this chapter we have provided an overview of the GNU/Linux operating system and its shell interface. Commonly used external tools were categorized, as well as most of the open-source languages in which tools are written. In particular we discussed GNU FORTRAN, GNU Ada, Java, Python, Tcl/Tk, Perl, Common Lisp, Scheme, Erlang, Smalltalk, Scala, X10, and Lua. PHP and Ruby are discussed in Chapter 20 (on web technologies). We have also presented an overview of remote computing with VNC.